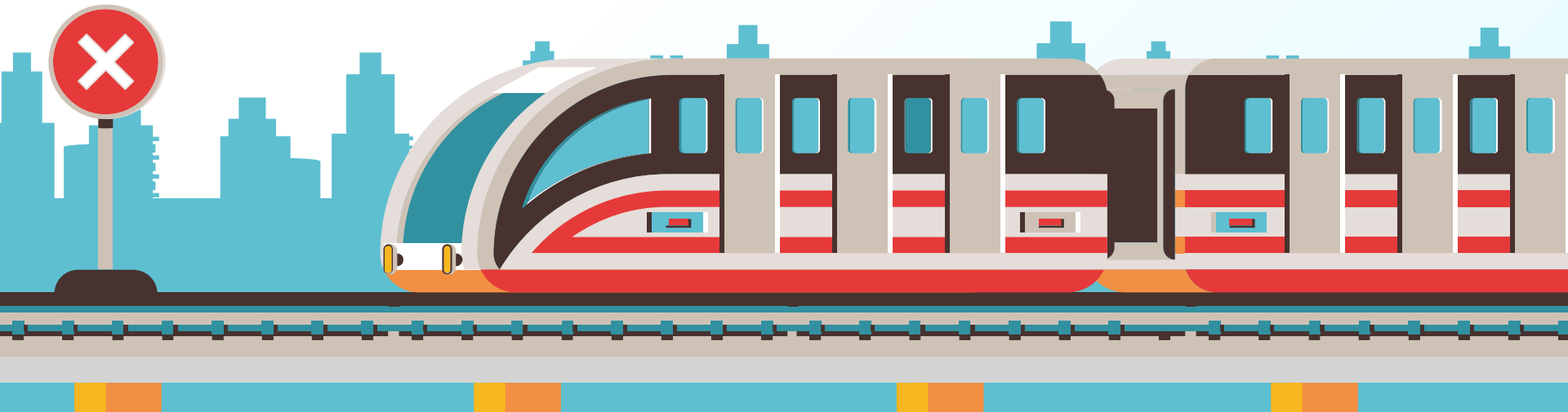
Generating Japanese Train Station Melodies Using Symbolic and Audio-Based Machine Learning

Shouki Katsuyama & Phillip Schiffman



Motivation

Japanese train stations use short departure melodies ("eki melodies") that are recognizable and musically distinct.

Research Questions:

- Can a model learn the structure of station melodies?
- Can generated outputs sound similar to authentic station jingles?
- How do symbolic and audio representations compare?

Features of Train Station Melodies:

repeated melody / limited number of instruments / short (5s to 30s) / unique common outro

Our Tasks

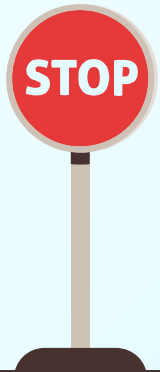
Task 1

Unconditional & Conditional generation of continuous audio representations of station jingles

Task 2

Conditional Symbolic generation of new station jingles using MIDI files (Did not work well)

Table of contents



01 Related Work

Related Work

Japanese railway companies use short departure melodies ("eki melodies") to:

- reduce passenger stress
- improve station identity
- provide auditory cues for train departures
- create recognizable local branding

Although there is not a widely-used academic machine learning dataset specifically for station jingles, similar collections have been used by:

- railway enthusiasts
- music archivists
- MIDI transcription communities
- online music sharing platforms

Related Work

Music Transformer

- Uses self attention instead of recurrence
- Advantages:
  - captures long-term structure
  - remembers motifs better
  - generates more coherent musical phrases

PerformanceRNN (Google Magenta)

- LSTM-based music generation
- learns note sequences directly from MIDI
- generates symbolic piano performances

Related Work

REMI Representation

- Popular MIDI tokenization strategy
- Represents the following as discrete tokens:
  - Pitch
  - Duration
  - Timing
  - Tempo
- Widely used in symbolic music generation

Jukebox

- OpenAI model that generates raw audio
- Advantages:
  - Realistic sound
- Disadvantages:
  - Extremely expensive to train

Related Work

How do we compare?

Similarities:

- Symbolic models successfully learned recurring melodic patterns from MIDI data
- MIDI generation was easier and more stable than raw audio generation
- Audio-to-MIDI transcription performed best on simple, monophonic melodies

Differences:

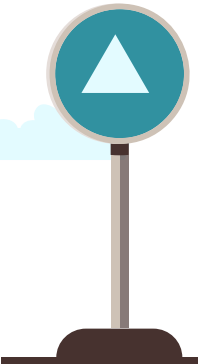
- We used a niche dataset of Japanese train station departure melodies rather than large benchmark datasets
- Our melodies were much shorter than typical audios used in other work
- Our evaluation relied more on listening tests and subjective musical quality rather than large-scale benchmark metrics

02 Exploratory Analysis

Data Collection & Processing

Data Collection

- Our data comes from two different sources:
  - Flat.io sheet music
  - A mp3 dropbox of JR Train Departure Melodies from a reddit thread
- 148 mp3 files from the dropbox
- Additional 65 mp3 files from sheet music
- Sheet music download had one large mp3 file, so we created split_audio.py
  - Splits mp3 file into smaller files based on two factors:
    - Repetition of a melody
    - Pause in music between melodies
- Some files from the dropbox were not melodies, so we manually verified them as well

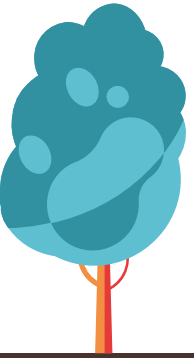


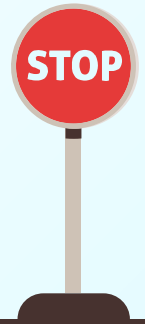
Data Pre-Processing (Task 1)

- Due to low amounts of data, we created our own augmentations
  - pitch_shift_down_4
  - pitch_shift_down_8
  - speed_up_125
  - background_noise
    - Ended up making quality worse so we scrapped this
  - slow_down
    - Also ended up taking out
- Did this by using librosa libraries
  - Allowed us to change the sample rate to match the models we used later on
- Set all files to length of the longest file (~28 seconds)

Data Pre-Processing (Task 2)

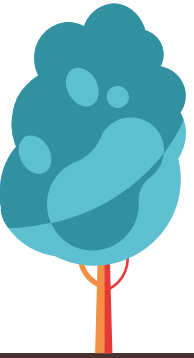
- Used **Spotify's Basic Pitch** model to convert mp3 files to MIDI
  - Creates per-file note event CSVs and summary CSVs as well as MIDI files
- **What it is**: a pretrained model + inference API that turns audio spectrograms into onset+pitch predictions, then groups those into note events and MIDI
- **Core steps**: decode MP3 -> resample/mono -> compute spectrogram -> model predicts per-frame pitch & onsets -> post-process frames into notes
- Ensured accuracy by manually listening to some of the MIDI files through a MIDI -> mp3 converter





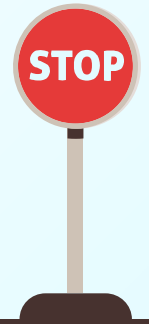
Condition Generations (Task 1/2)

- We do not have corresponding conditions. (only audio files)
- Train melodies are sometimes related to characteristics of stations but most melodies are used in multiple stations.
- Conditional generation works generally better than unconditional one (proved by a lot of image generation models).



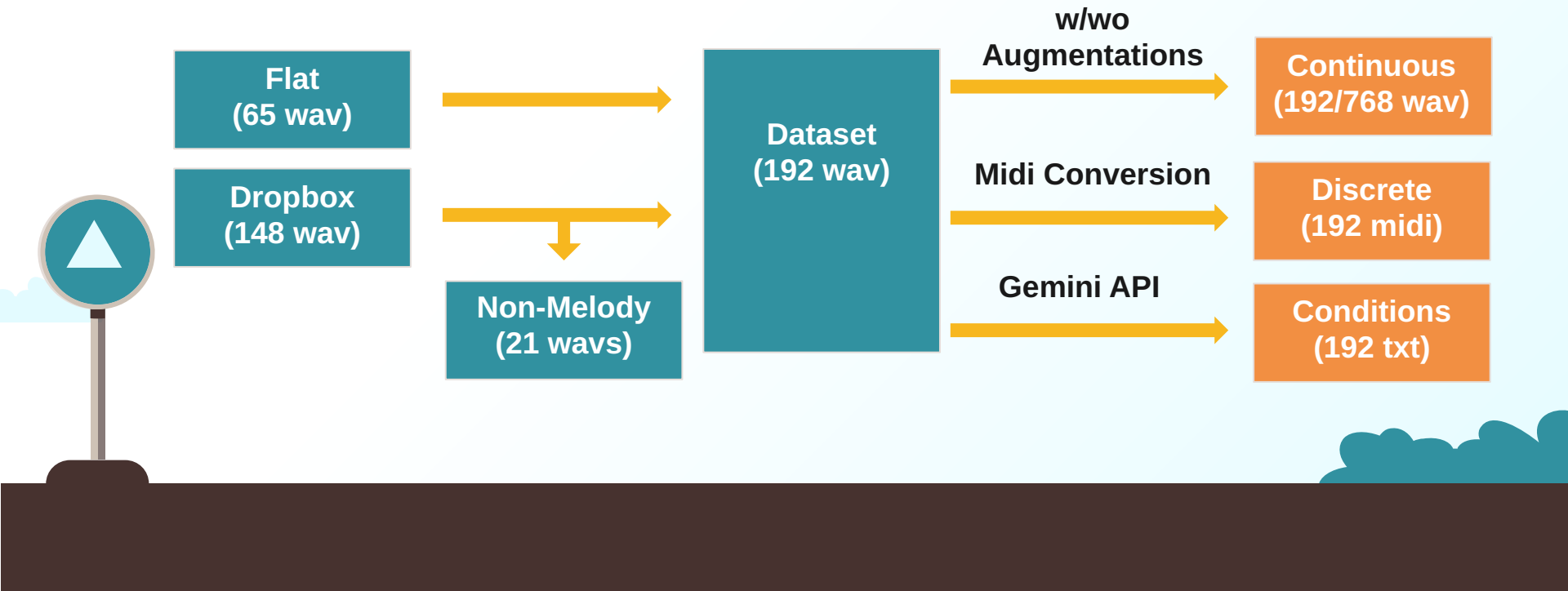
Use Gemini API to Generate Descriptions of Audio Files

(Generating dataset with LLM is very common approach)



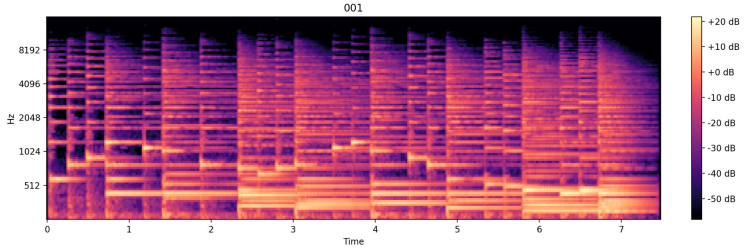
Pipeline

Generating train station jingles in continuous (wav) and discrete (midi) domain

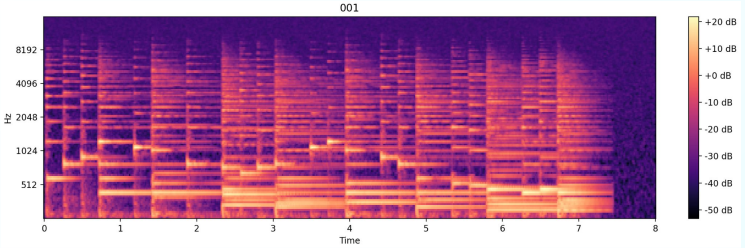


Spectrograms of Processed Data

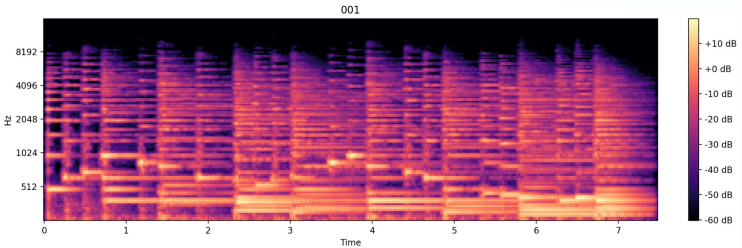
Normal Processed Spectrogram



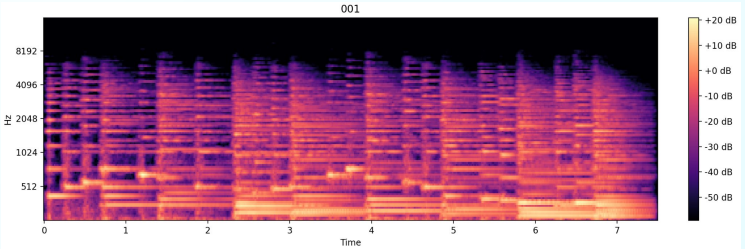
Background Processed Spectrogram



Pitch Shift Down 4 Spectrogram



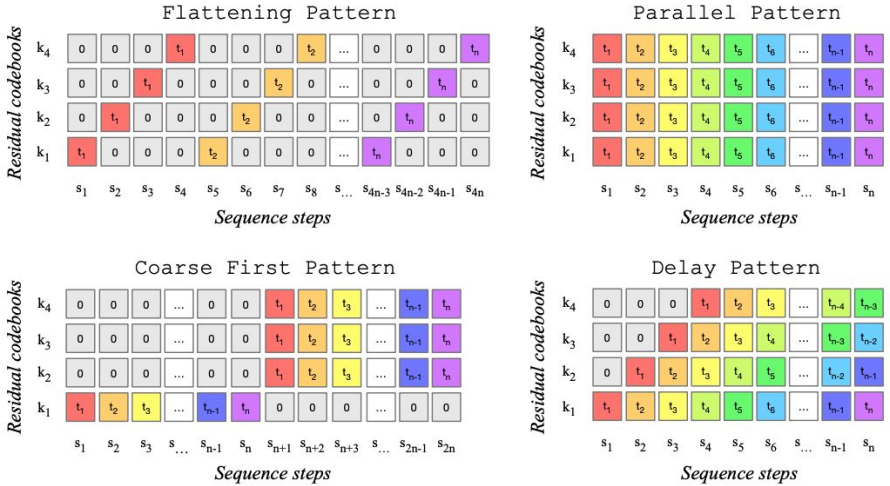
Pitch Shift Down 8 Spectrogram



03 Modeling

Task 1: MusicGen

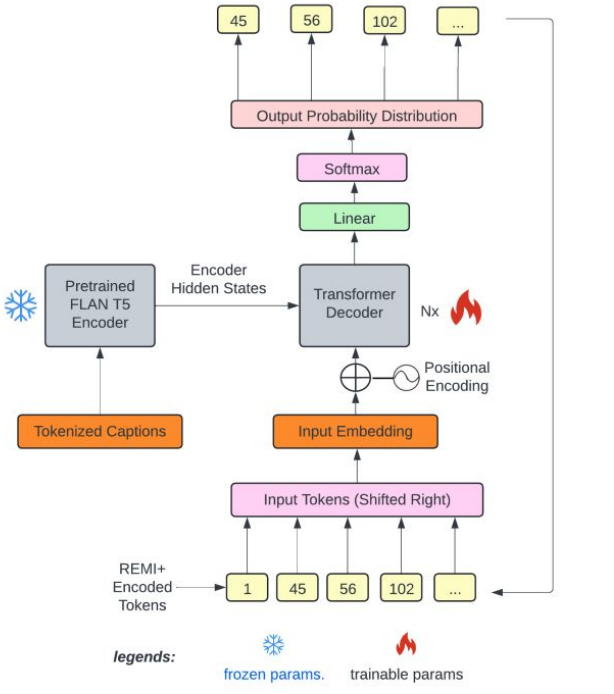
https://arxiv.org/pdf/2306.05284



- A model released in 2023 by Meta
- It is easy to use with AudioCraft library
- Codebook based mode:
  Generate a codebook from audio then use Transformer to model sequences of codebooks.

Task 2: Text2Midi

https://arxiv.org/abs/2412.16526



- Pretrained model for text to midi generation released in 2024.
- This model is encoder-decoder Transformer model.
- MidiCaps was used for pretraining.
- It is difficult to do unconditional generation because condition is required rather than optional.

Why not Diffusion/Flow Models?

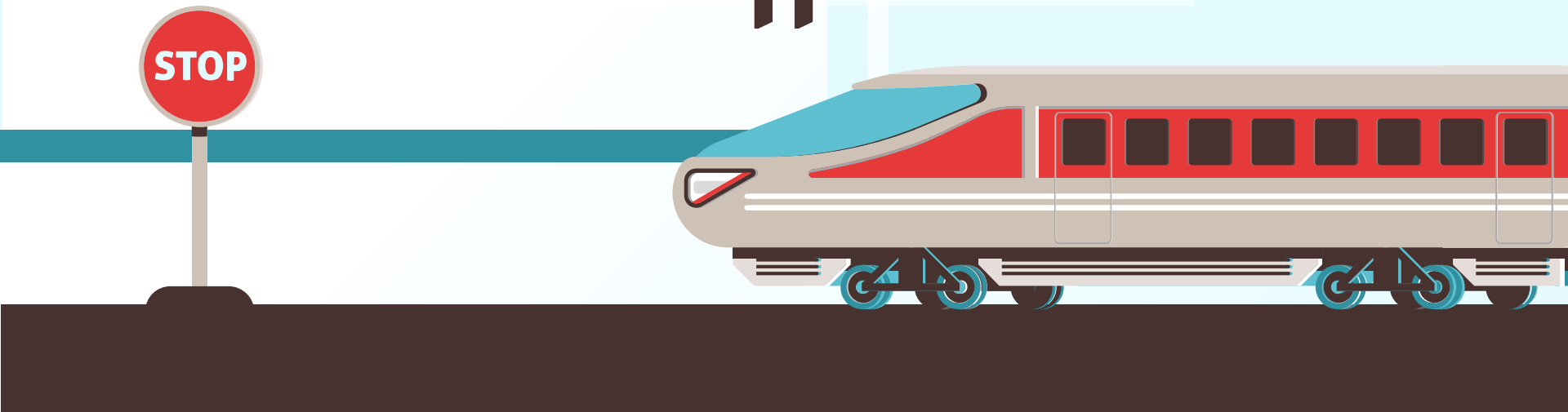
Implementations

- AudioCraft allow us to easily implement MusicGen.
- Text2midi was using torch Module so it was easy to modify/finetune.

Complexity (Task is Simple)

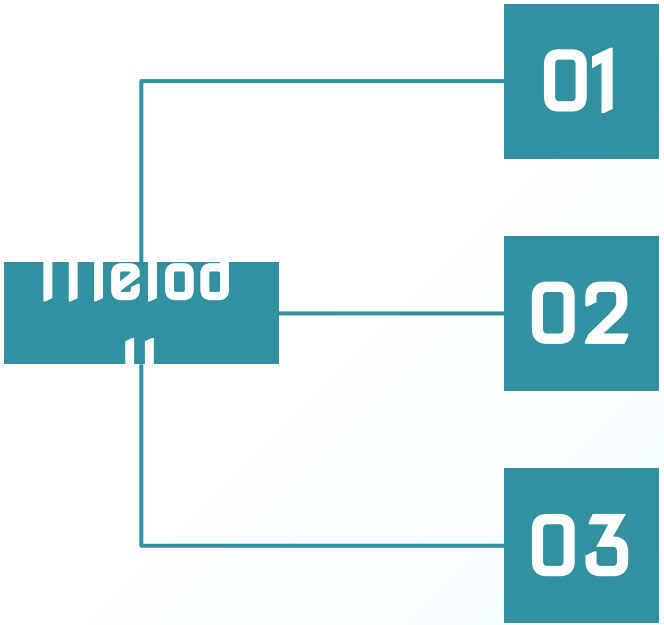
- Even training one model takes a lot of time, so it would be better to focus on one model with different sizes and hyperparameters.
- Diffusion model requires more inference time than Transformers.

Code Walkthrough



04 Evaluation

Challenges of Evaluation



Subjective

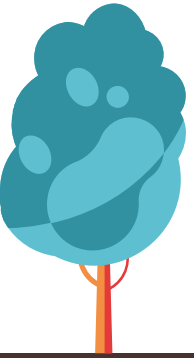
How it sound like actual train station jingle is totally subjective

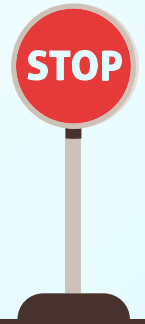
Not Only About Quality

Some train jingle has very simple but very catchy and useful melody

Limited Dataset

It is not ideal to use some of them just for testing. (only 1500 samples)





Augmentations / Conditions

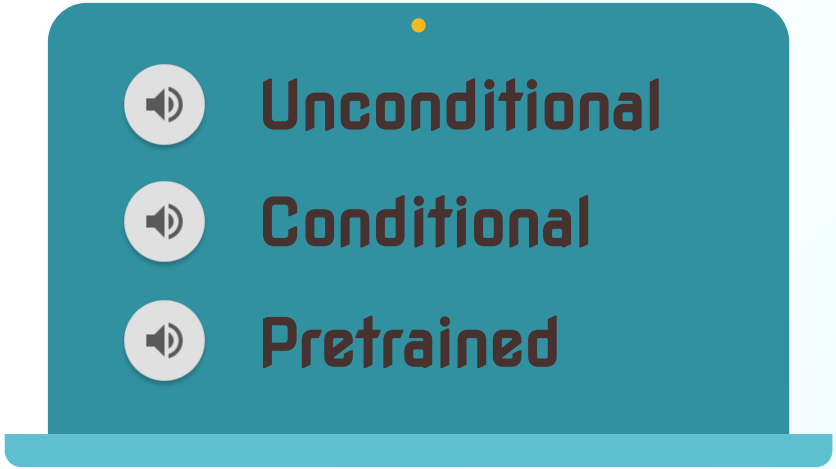
Augmentations

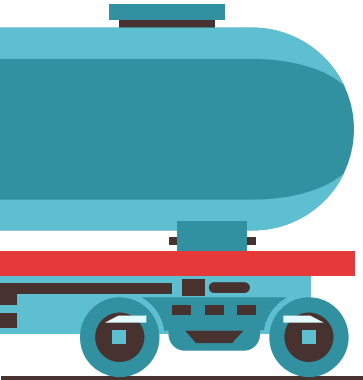
- Augmentation of adding background noise clearly worked negatively and generated samples started having noises.
- Raw dataset already has some samples with high pitch, shifting pitches up did not work well.
- Augmentation did not work well and resulted in overfitting (more same samples per epoch).

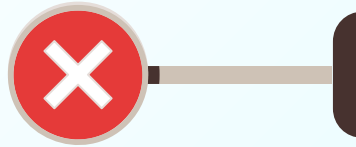
Conditions

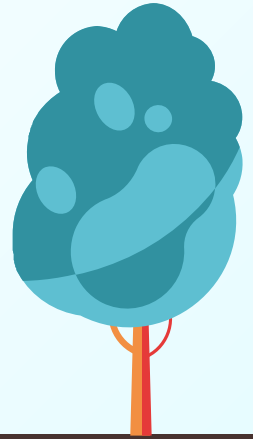
- Conditioned samples generally sound better than unconditioned ones.
- You can see that conditions actually influencing generations.

Demo: Generated Melodies



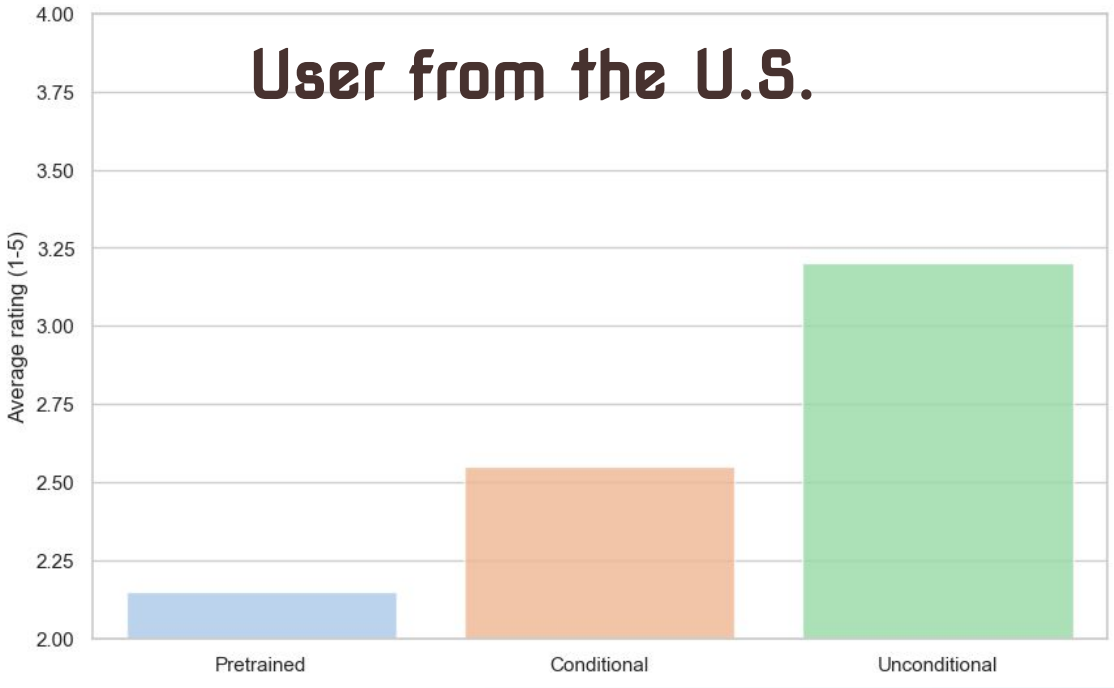






You can hear the unique features of train station melodies!!

Human Evaluation for Task 1



Pretrained

A model before finetuning with train station jingles.

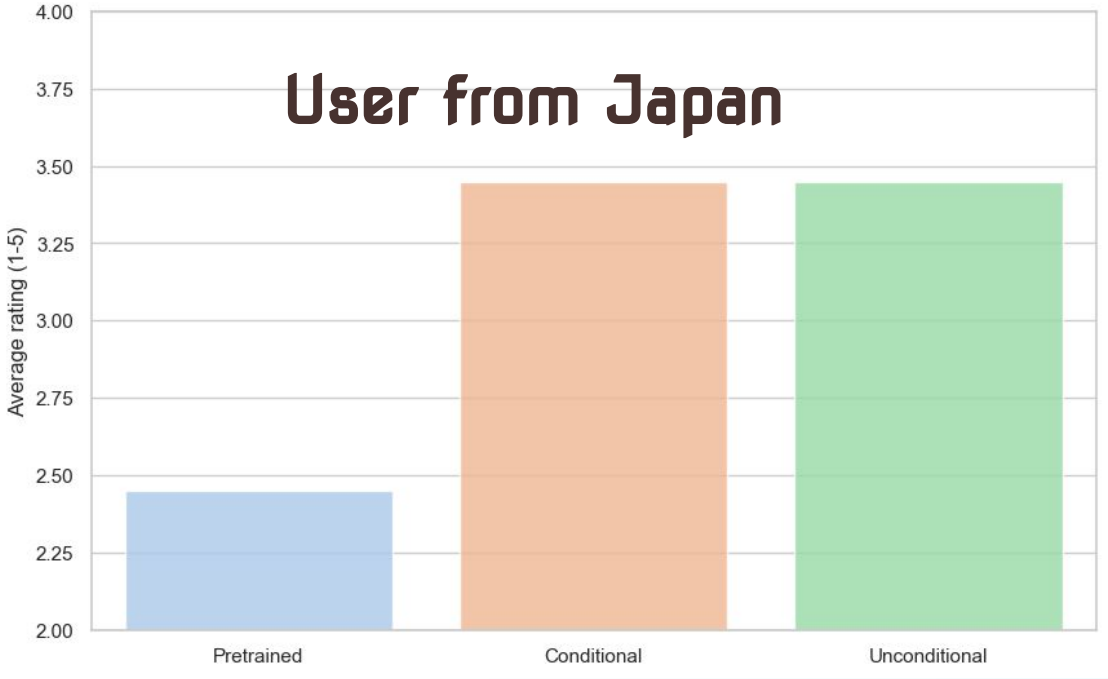
Conditional

A model finetuned with conditional inputs

Unconditional

A model finetuned without conditional inputs

Human Evaluation for Task 1



Pretrained

A model before finetuning with train station jingles.

Conditional

A model finetuned with conditional inputs

Unconditional

A model finetuned without conditional inputs

Failure of Task 2

Why did it fail?

Pretrained Model

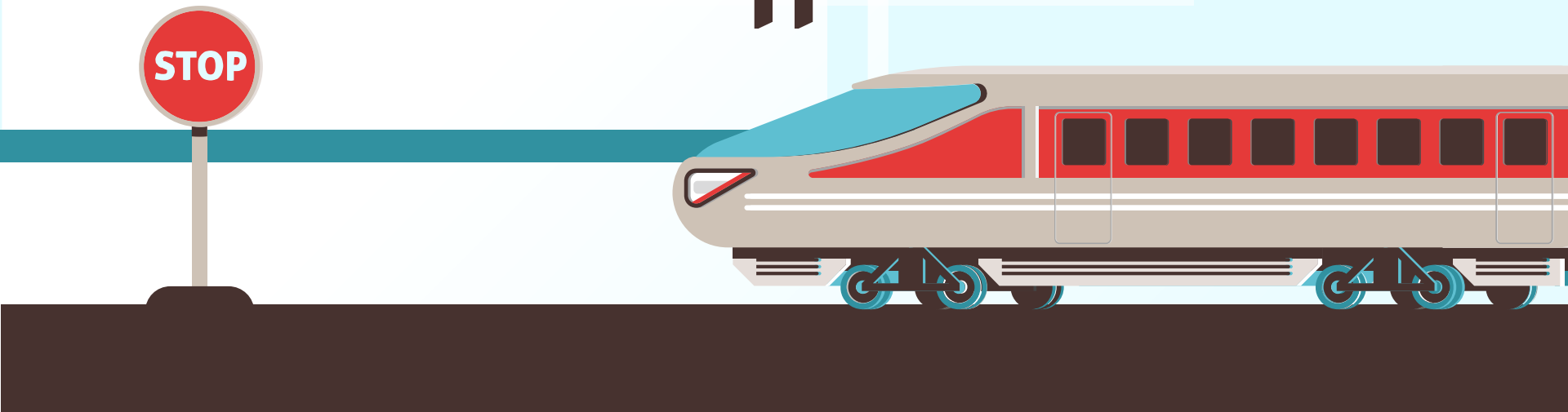
Pretrained model “text2midi” itself is not that good (you can see from generations before training)

Wav to Midi

Wav to midi conversion with a model from Spotify was not a good quality

Trying with more complex models would be our future works!

Code Walkthrough



Thanks!

CREDITS: This presentation template was created by **Slidesgo**, and includes icons by **Flaticon**, and infographics & images by **Freepik**